

Prüfungsfragen Nini - Modul 347 - 30 Competencies

A - Grundlagen

A1G Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:18 PM • Reviewed Apr 17, 2026, 02:41 PM

Single Choice Question:

Was ist ein zentraler Unterschied zwischen Containern und virtuellen Maschinen?

- ☐ A Container benötigen immer ein eigenes Betriebssystem
- ☐ B Virtuelle Maschinen teilen sich den Kernel des Host-Systems
- ☒ C Container virtualisieren auf Betriebssystem-Ebene
- ☐ D Virtuelle Maschinen starten schneller als Container

A1F Approved Difficulty: Medium

Submitted May 22, 2026, 02:05 PM • Reviewed May 22, 2026, 02:15 PM

Multiple Choice Question:

Welche Anforderungen sprechen typischerweise für den Einsatz von Containern bei einem IT-Dienst?

- ☒ A Konsistente Laufzeitumgebungen
- ☒ B Schnelle Bereitstellung und Startzeit
- ☐ C Vollständige Virtualisierung eines Betriebssystems pro Dienst
- ☐ D Einfache Skalierung mehrerer Instanzen
- ☒ E Direkter Zugriff auf physische Hardware ohne Isolation

A1E Approved Difficulty: Hard

Submitted Apr 17, 2026, 02:19 PM • Reviewed May 22, 2026, 02:03 PM

Question:

Nenne zwei Schwachstellen eines Dienstes, bei dem Frontend und Backend gemeinsam in einem Container betrieben werden, und erkläre kurz, wie diese durch eine Container-Aufteilung verbessert werden können.

Answer/Solution:

Durch die gemeinsame Ausführung von Frontend und Backend wird die Wartbarkeit erschwert, da jede Änderung den gesamten Container betrifft. Zudem ist die Skalierung eingeschränkt, da nicht einzelne Komponenten separat skaliert werden können. Durch eine Aufteilung in mehrere Container können Komponenten unabhängig aktualisiert und gezielt skaliert werden.

B1 - Containerkomposition (Architektur)

B1G Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:21 PM • Reviewed May 22, 2026, 02:03 PM

Single Choice Question:

Welche Aussage beschreibt eine monolithische Architektur korrekt?

- ☐ A Frontend, Backend und Datenbank laufen in getrennten Containern
- ☐ B Jeder Service wird unabhängig deployed
- ☒ C Die gesamte Anwendung läuft in einem einzigen Container
- ☐ D Ein Auth-Service wird separat betrieben

B1F Approved Difficulty: Medium

Submitted Apr 17, 2026, 02:21 PM • Reviewed May 22, 2026, 02:03 PM

Question:

Warum ist bei einer Web-Applikation die Trennung in Frontend-, Backend- und Datenbank-Container sinnvoll? Nenne zwei Gründe und erkläre sie kurz.

Answer/Solution:

Durch die Trennung der Komponenten können Frontend und Backend unabhängig voneinander gewartet und aktualisiert werden. Zudem kann das Backend bei hoher Last separat skaliert werden, ohne das Frontend zu vervielfältigen. Die Datenbank bleibt isoliert, was die Stabilität und Sicherheit erhöht.

B1E Approved Difficulty: Medium

Submitted Apr 17, 2026, 02:23 PM • Reviewed May 22, 2026, 02:05 PM

Multiple Choice Question:

Welche Aussagen treffen auf eine erweiterte Microservice-Architektur zu?

- ☒ A Jeder Service kann separat skaliert werden
- ☐ B Die Architektur ist weniger komplex als ein Monolith
- ☒ C Zusätzliche Services wie ein Auth-Service sind möglich
- ☒ D Der Betriebs- und Konfigurationsaufwand ist höher
- ☐ E Es wird immer nur ein Container verwendet

B2

B2G Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:24 PM • Reviewed May 22, 2026, 02:06 PM

Multiple Choice Question:

Welche der folgenden Bestandteile gehören zur Container-Technologie und werden beim Arbeiten mit Docker verwendet?

- ☒ A Docker Engine
- ☒ B Dockerfile
- ☒ C Container Image
- ☐ D Hypervisor
- ☒ E Container Registry

B2F Approved Difficulty: Hard

Submitted May 22, 2026, 02:12 PM • Reviewed May 22, 2026, 02:15 PM

Question:

Ein Entwickler erstellt eine containerisierte Webanwendung mit Frontend, Backend und Datenbank.

Beschreibe:

1. in welcher Reihenfolge die Anwendung typischerweise entwickelt und betrieben wird
2. warum docker compose für die lokale Entwicklung sinnvoll ist
3. welche Probleme ohne gemeinsames lokales Testen auftreten könnten

Answer/Solution:

Zuerst wird die Anwendung entwickelt und ein Dockerfile erstellt. Danach wird ein Container Image gebaut. Anschliessend werden mehrere Services mit docker compose gemeinsam gestartet und getestet. Danach kann die Anwendung auf einem Server oder in der Cloud betrieben werden. docker compose ermöglicht das gemeinsame lokale Starten mehrerer Services wie Frontend, Backend und Datenbank. Dadurch kann das Zusammenspiel früh getestet werden. Ohne gemeinsames lokales Testen können Probleme zwischen den Services erst später im Betrieb auffallen, z. B. falsche Ports, Netzwerkprobleme oder fehlende Verbindungen zur Datenbank.

B2E Approved Difficulty: Easy

Submitted May 22, 2026, 02:16 PM • Reviewed Jun 5, 2026, 02:22 PM

Single Choice Question:

Betrachte folgendes vereinfachtes Netzwerkdiagramm:

Warum sollte in diesem Aufbau nur der Reverse Proxy direkt aus dem Internet erreichbar sein?

Image:



- ☐ A Damit Backend und Datenbank schneller laufen
- ☒ B Damit nur notwendige Ports öffentlich erreichbar sind und interne Services geschützt bleiben
- ☐ C Damit alle Container dieselbe IP-Adresse nutzen
- ☐ D Damit keine Kommunikation zwischen den Containern möglich ist

C1 - Containerdienstleister auswählen

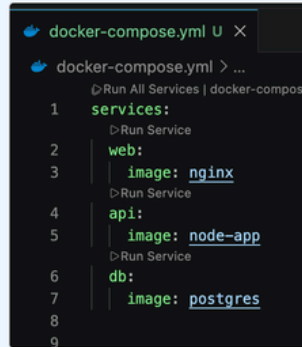
C1G Approved Difficulty: Hard

Submitted Apr 17, 2026, 02:47 PM • Reviewed May 22, 2026, 02:11 PM

Question:

Welche zwei zusätzlichen Dienste sollte ein Containerdienstleister bereitstellen, damit diese Anwendung produktiv betrieben werden kann? Begründe kurz.

Image:



```
docker-compose.yml U X
docker-compose.yml > ...
Run All Services | docker-compose
1 services:
  Run Service
2 web:
  image: nginx
  Run Service
3 api:
  image: node-app
  Run Service
4 db:
  image: postgres
5
6
7
8
9
```

Answer/Solution:

Load Balancer → verteilt Anfragen auf mehrere Instanzen (bessere Verfügbarkeit und Performance)
Container Registry → speichert und verteilt Images zentral
(optional) Orchestrierung (z. B. Kubernetes) → automatisches Deployment und Skalierung

C1F Approved Difficulty: Medium

Submitted Apr 17, 2026, 02:48 PM • Reviewed May 22, 2026, 02:11 PM

Multiple Choice Question:

Welche Aussagen über Public Cloud Anbieter sind korrekt?

- ☒ A Ressourcen können dynamisch skaliert werden
- ☒ B Man hat vollständige physische Kontrolle über Server
- ☒ C Dienste sind meist weltweit verfügbar
- ☐ D Kosten entstehen oft nutzungsabhängig

C1E Approved Difficulty: Hard

Submitted Jun 19, 2026, 11:50 AM • Reviewed Jun 19, 2026, 11:51 AM

Question:

Betrachte die Vergleichstabelle zwischen Google Kubernetes Engine (GKE), Microsoft Azure Container Service (ACS) und Amazon Elastic Container Service (ECS).

Erstelle anhand der folgenden Gewichtung eine Entscheidungsmatrix und bestimme den geeignetsten Containerdienstleister für ein Unternehmen, das grossen Wert auf Sicherheit, Monitoring und DevOps-Integration legt.

Kriterium	Gewichtung
Security	5
Monitoring	4
DevOps Tool Integrationen	4
Pricing	2
Service Discovery	3

Bewerte jeden Anbieter pro Kriterium mit 1 bis 5 Punkten und berechne anschliessend die Gesamtpunktzahl.

Welcher Anbieter erreicht die höchste Punktzahl?

Quelle: <https://pablo-iorio.medium.com/container-based-architectures-iii-iii-public-cloud-providers-options-aws-azure-and-google-708667198b5e>

Image:

	Google Google Container Engine (GKE)	Microsoft Azure Container Service	Amazon Amazon Elastic Container Service (ECS)
Orchestration (container scheduling, cluster management, and provisioning)	Kubernetes	Marathon and DCOS, Docker Swarm, or Kubernetes	Based on a proprietary clustering technology
Pricing	Cluster Management: Free up to 5 nodes, more than 5 nodes Google charges \$0.10 per cluster per hour. Nodes instances: You will be billed for these instances according to Compute Engine's pricing, until the nodes are deleted.	Cluster management: ACS is a free service. Nodes instances: You pay for the VM instances associated storage and networking resources consumed.	Cluster management: Amazon ECS Container Service is a fee. Nodes instances: You pay for AWS resources (e.g. EC2 instances or EBS volumes) you create to store and run your application.
Security	Leverage Kubernetes security features. Control access in the cluster with your Google accounts and role permissions.	Each orchestrator available in Azure Container Service has its own security considerations.	Parameter Store and task IAM roles or custom integration with Vault or KeyOps. Encryption of secrets with custom keys with AWS.
DevOps tool integrations	Google Container Builder, CircleCI, Codefresh, Jenkins, Semaphore, Shipshape, Spinnaker, TeamCity, Wercker, Cloud Build, Google Container Registry	Azure Container Registry, Azure CLI, Visual Studio Team Services, Jenkins, Spinnaker, Spinnaker, TeamCity	AWS CodePipeline, AWS CodeBuild, and AWS CloudFormation, Amazon ECS Container Registry, AWS CLI, CircleCI, Codefresh, Jenkins, Semaphore, Shipshape, Spinnaker, TeamCity, Wercker
Monitoring	Google Cloud Monitoring, InfluxDB and Grafana, Prometheus, Stackdriver, OpenTSDB, Kibana, Riemann, ELK, Prometheus	ELK, OMS, Datadog, Sysdig, Dynatrace, CloudWatch, Prometheus	CloudWatch, Datadog, statsd, ELK, Prometheus, Grafana, Grafana
Service Discovery	Only requests from outside the cluster are passing through a load balancer. A virtual IP provides access to internal services without the need for a load balancer.	Azure uses load balancers for discovery. Azure Load Balancer provides public entry points. The Microsoft Load Balancer (mshlb) makes inbound requests to container instances that service these requests.	ECS is using load balancers for service discovery. External as well as internal services are accessible through load balancers. The Application Load Balancer (ALB) offers path- and host-based routing as well as internal or external connections.

Kriterium	Beschreibung	Gewichtung	Begründung
Security	Schutzmechanismen wie Rollen, Berechtigungen, Verschlüsselung und Secrets-Management.	5	Sicherheit ist für produktive Container-Anwendungen besonders wichtig, da Sicherheitslücken zu Datenverlust oder unbefugtem Zugriff führen können.
Monitoring	Möglichkeiten zur Überwachung von Containern, Diensten und Ressourcen.	4	Fehler und Leistungsprobleme sollen früh erkannt werden, weshalb Monitoring eine hohe Bedeutung hat.
DevOps Tool Integrationen	Unterstützung von CI/CD-Werkzeugen, Registries und Automatisierungstools.	4	Eine gute Integration vereinfacht Entwicklung, Deployment und Wartung der Anwendung.
Pricing	Kosten für den Betrieb des Containerdienstes.	2	Die Kosten sind relevant, stehen bei diesem Szenario aber hinter Sicherheit und Funktionalität zurück.
Service Discovery	Mechanismen, mit denen Container und Services automatisch gefunden und verbunden werden können.	3	Wichtig für verteilte Anwendungen, aber weniger kritisch als Sicherheit oder Monitoring.

Answer/Solution:

GKE
 $(5 \times 5) + (4 \times 5) + (4 \times 5) + (2 \times 4) + (3 \times 4)$
 $= 25 + 20 + 20 + 8 + 12$
 $= 85 \text{ Punkte}$

Azure
 $(5 \times 3) + (4 \times 4) + (4 \times 4) + (2 \times 4) + (3 \times 4)$
 $= 15 + 16 + 16 + 8 + 12$
 $= 67 \text{ Punkte}$

AWS
 $(5 \times 5) + (4 \times 5) + (4 \times 5) + (2 \times 4) + (3 \times 5)$
 $= 25 + 20 + 20 + 8 + 15$
 $= 88 \text{ Punkte}$

Image:

Kriterium	Gewicht	GKE	Azure	AWS
Security	5	5	3	5
Monitoring	4	5	4	5
DevOps	4	5	4	5
Pricing	2	4	4	4
Service Discovery	3	4	4	5

C2

C2G Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:28 PM • Reviewed May 22, 2026, 02:07 PM

Single Choice Question:

Was ist die Hauptaufgabe einer Container Registry?

- ☐ A Container direkt auszuführen
- ☐ B Container automatisch zu skalieren
- ☐ C Netzwerke zwischen Containern zu konfigurieren
- ☒ D Container Images zu speichern und versioniert bereitzustellen

C2F Approved Difficulty: Medium

Submitted Apr 17, 2026, 02:29 PM • Reviewed May 22, 2026, 02:07 PM

Multiple Choice Question:

Welche Technologien dienen der Orchestrierung von Containern?

- ☒ A Docker Compose
- ☒ B Kubernetes
- ☒ C Docker Swarm
- ☐ D NGINX
- ☐ E GitHub Actions

C2E Approved Difficulty: Hard

Submitted Apr 17, 2026, 02:30 PM • Reviewed May 22, 2026, 02:07 PM

Question:

Beschreibe in eigenen Worten zwei Unterschiede zwischen einer public und einer private Container Registry und erkläre, warum eine private Registry für interne Projekte besser geeignet ist.

Answer/Solution:

Public Registry: frei zugänglich, kein Login nötig
Private Registry: Zugriff nur mit Authentifizierung
Private Registry bietet mehr Sicherheit und Kontrolle

D1 - Praxis mit Containern in der Entwicklung erlangen

D1G Approved Difficulty: **Hard**

Submitted May 22, 2026, 02:24 PM - Reviewed Jun 5, 2026, 02:23 PM

Question:

Du möchtest mit Docker Desktop einen einfachen NGINX-Webdienst als Container starten und im Browser auf Port 80 testen. Der Container soll den Namen web-demo heissen und von Port 8080 gemappt werden.

Nenne die wichtigsten Docker-Befehle in der richtigen Reihenfolge und erkläre kurz, was jeder Befehl macht.

Aufgaben:

- image laden
- laufen lassen
- läutert er?
- Logs anzeigen
- stoppen
- löschen

Answer/Solution:

```
docker pull nginx
// Lädt das NGINX-Image aus der Registry herunter.

docker run -d --name web-demo -p 8080:80 nginx
// Startet den Container im Hintergrund, gibt ihm den Namen web-demo und mappt Port 8080 vom Host auf Port 80 im Container.

docker ps
// Zeigt, ob der Container läuft.

docker logs web-demo
// Zeigt die Logs des Containers.

docker stop web-demo
// Stoppt den Container.

docker rm web-demo
// Löscht den Container.
```

D1F Approved Difficulty: **Hard**

Submitted May 22, 2026, 02:26 PM - Reviewed Jun 5, 2026, 02:24 PM

Question:

Erstelle eine einfache docker-compose.yml für einen Webdienst mit NGINX.

Die Konfiguration soll folgende Anforderungen erfüllen:

Service heisst web
Image: nginx:alpine
Containername: d1f-web
Port 8080 vom Host wird auf Port 80 im Container gemappt
Der Container wird automatisch neu gestartet, ausser er wurde manuell gestoppt

Answer/Solution:

Siehe Bild:

```
services:
  web:
    image: nginx:alpine
    container_name: d1f-web
    ports:
      - "8080:80"
    restart: unless-stopped
```

Image:

```
docker-compose.yml > {} services > {} web > abc restart
docker-compose.yml - The Compose specification establishes a standard
1  services:
   ↳ Run Service
2  web:
3    image: nginx:alpine
4    container_name: d1f-web
5    ports:
6    - "8080:80"
7    restart: unless-stopped
```

D1E Approved Difficulty: **Medium**

Submitted May 22, 2026, 02:28 PM - Reviewed Jun 5, 2026, 02:24 PM

Question:

In der folgenden docker-compose.yml startet der Webdienst nicht korrekt.

Finde mindestens drei Fehler und korrigiere die Konfiguration.

```
services:
  web:
    image: nginx:alpine
    container_name: d1e-web
    port:
      - "8080"
    volumes:
      - ./wrong-index.html:/usr/share/nginx/html/index.html
    enviroment:
      - APP_ENV=dev
```

Answer/Solution:

Fehler
port ist falsch, korrekt ist ports
Port-Mapping ist unvollständig, korrekt z. B. "8080:80"
enviroment ist falsch geschrieben, korrekt ist environment
Der Volume-Pfad ./wrong-index.html kann falsch sein, falls die Datei nicht existiert
Image:

```
docker-compose.yml > {} services > {} web > {} environment > abc 0
1  services:
   ↳ Run Service
2  web:
3    image: nginx:alpine
4    container_name: d1e-web
5    ports:
6    - "8080:80"
7    volumes:
8    - ./index.html:/usr/share/nginx/html/index.html
9    environment:
10   - APP_ENV=dev
```

D2

D2G Approved Difficulty: **Medium**

Submitted Apr 17, 2026, 02:49 PM • Reviewed May 22, 2026, 02:11 PM

Question:

Gegeben ist folgender Befehl:

```
docker run -d -p 5000:80 --name web nginx
```

Nenne zwei Dinge, die dieser Befehl bewirkt, und erkläre kurz deren Zweck.

Answer/Solution:

Container läuft im Hintergrund (-d) → Terminal bleibt frei
Port 5000 (Host) wird auf Port 80 (Container) gemappt → Anwendung ist erreichbar
Container erhält den Namen web → einfachere Verwaltung

D2F Approved Difficulty: **Medium**

Submitted May 22, 2026, 02:30 PM • Reviewed Jun 5, 2026, 02:24 PM

Question:

Ein Entwickler möchte eine eigene Webanwendung containerisieren und anschliessend in eine Registry publizieren.

Gegeben ist folgendes Dockerfile:

Beschreibe die notwendigen Schritte und Docker-Befehle, um daraus ein eigenes Image zu erstellen und in eine Registry hochzuladen.

Image:

```
Dockerfile > ...
1 FROM nginx:alpine
2
3 COPY ./website /usr/share/nginx/html
4
5 EXPOSE 80
6
```

Answer/Solution:**1. Dockerfile erstellen**

Das Dockerfile beschreibt, wie das Image aufgebaut wird.

2. Image bauen

```
docker build -t my-webapp .
```

Erstellt ein eigenes Docker Image.

3. Image für Registry taggen

```
docker tag my-webapp ghcr.io/user/my-webapp:latest
```

Bereitet das Image für die Registry vor.

4. Login bei der Registry

```
docker login ghcr.io
```

Authentifiziert den Benutzer.

5. Image publizieren

```
docker push ghcr.io/user/my-webapp:latest
```

Lädt das Image in die Registry hoch.

D2E Approved Difficulty: **Medium**

Submitted Apr 17, 2026, 02:50 PM • Reviewed May 22, 2026, 02:12 PM

Question:

Warum kann es sinnvoll sein, ein Container-Image in mehrere Registries zu publizieren? Nenne einen Vorteil.

Answer/Solution:

Höhere Verfügbarkeit → falls eine Registry ausfällt, ist das Image weiterhin verfügbar
Flexibilität → bessere Integration mit verschiedenen Plattformen

E1 - Praxis mit Containern beim Public-Cloud-Anbieter (PCA) erlangen (Betrieb)

E1G Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:51 PM • Reviewed May 22, 2026, 02:12 PM

Single Choice Question:

Welche Massnahme erhöht die Sicherheit beim Zugriff auf einen Cloud-Anbieter am meisten?

- ☐ A Langes Passwort verwenden
- ☐ B Zugriff ohne Login ermöglichen
- ☒ C Multi-Faktor-Authentifizierung (MFA) aktivieren
- ☐ D Zugriff über HTTPS statt HTTP

E1F Approved Difficulty: Medium

Submitted May 22, 2026, 02:32 PM • Reviewed Jun 5, 2026, 02:24 PM

Question:

Ein Entwickler erstellt eine Cloud-Instanz für einen containerisierten Webdienst.

Folgende Einstellungen wurden konfiguriert:

Name: web-server-01
 Betriebssystem: Ubuntu 24.04
 Öffentliche IP: aktiviert
 Firewall:
 - Port 22 offen
 - Port 80 offen

Erkläre die Funktion von Port 22 und Port 80 und beschreibe, warum diese Konfiguration für einen Webserver sinnvoll ist.

Answer/Solution:

Port 22 wird für SSH verwendet und ermöglicht sicheren Remote-Zugriff auf die Cloud-Instanz zur Administration.
 Port 80 wird für HTTP verwendet und ermöglicht den Zugriff auf den Webdienst über den Browser.
 Die öffentliche IP erlaubt den externen Zugriff auf den Server.
 Die Firewall erlaubt nur die benötigten Ports und erhöht dadurch die Sicherheit.

E1E Approved Difficulty: Medium

Submitted May 22, 2026, 02:33 PM • Reviewed Jun 5, 2026, 02:25 PM

Question:

Eine containerisierte Anwendung soll gemäss folgenden Vorgaben betrieben werden:

Nur der Reverse Proxy darf öffentlich erreichbar sein
 Das Backend darf nur intern erreichbar sein
 Die Datenbank darf ausschliesslich mit dem Backend kommunizieren

Gegeben ist folgendes vereinfachtes Netzwerkdiagramm:

Beschreibe zwei Sicherheitsmassnahmen, die bei der Netzwerk-Konfiguration umgesetzt werden sollten, damit diese Vorgaben eingehalten werden.

Image:



Answer/Solution:

Nur der Reverse Proxy sollte öffentliche Ports (80/443) besitzen.
 Backend und Datenbank dürfen keine öffentlichen Ports freigeben.
 Die Kommunikation zwischen Backend und Datenbank sollte über ein internes Docker-Netzwerk erfolgen.
 Die Datenbank sollte ausschliesslich Verbindungen vom Backend akzeptieren.

E2

E2G Approved Difficulty: **Medium**

Submitted Apr 17, 2026, 02:53 PM • Reviewed May 22, 2026, 02:13 PM

Single Choice Question:

Welche Storage-Variante eignet sich für persistente Daten einer Datenbank?

- ☐ A Temporärer Container-Speicher
- ☒ B Persistent Volume
- ☐ C RAM
- ☐ D Log-Datei

E2F Approved Difficulty: **Easy**

Submitted Apr 17, 2026, 02:54 PM • Reviewed May 22, 2026, 02:13 PM

Multiple Choice Question:

Welche Möglichkeiten zur Skalierung gibt es in der Cloud?

- ☒ A Vertikale Skalierung
- ☒ B Horizontale Skalierung
- ☐ C Löschen von Instanzen
- ☒ D Auto-Scaling

E2E Approved Difficulty: **Medium**

Submitted May 22, 2026, 02:34 PM • Reviewed Jun 5, 2026, 02:25 PM

Question:

Ein Entwickler liest auf einer Cloud-Instanz folgende Informationen aus:

```
{  
  "instance-id": "vm-01",  
  "region": "europe-west6",  
  "public-ip": "34.65.10.20"  
}
```

Welche Art von Informationen werden hier angezeigt und wofür können sie verwendet werden?

Answer/Solution:

Es handelt sich um Metadaten der Cloud-Instanz.

Diese Informationen können verwendet werden für:

Identifikation der Instanz (instance-id)
Standort / Region der VM (region)
Netzwerkzugriff über die öffentliche IP (public-ip)

Metadaten helfen bei Verwaltung, Monitoring und automatisierter Konfiguration von Cloud-Instanzen.

F - Qualitätskontrollen planen und umsetzen

F1G Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:55 PM • Reviewed May 22, 2026, 02:14 PM

Question:

Nenne eine Methode zur Qualitätssicherung von Container-Images und erkläre kurz.

Answer/Solution:

Image Scanning (z. B. Trivy) → überprüft Images auf Sicherheitslücken und veraltete Pakete

F1F Approved Difficulty: Easy

Submitted Apr 17, 2026, 02:56 PM • Reviewed May 22, 2026, 02:14 PM

Multiple Choice Question:

Welche Punkte sind wichtige Sicherheitsindikatoren für Container? (Mehrere Antworten möglich)

- ☒ A Verwendung aktueller Base Images
- ☐ B Container läuft als root
- ☒ C Minimale Anzahl offener Ports
- ☒ D Verwendung von Secrets statt Klartext-Passwörtern

F1E Approved Difficulty: Medium

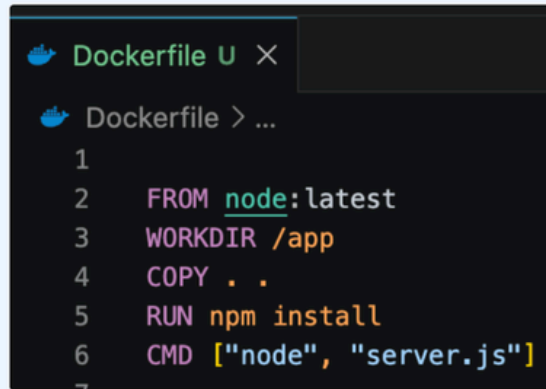
Submitted May 22, 2026, 02:36 PM • Reviewed Jun 5, 2026, 02:25 PM

Question:

Gegebenes Dockerfile:

Nenne zwei Sicherheitsprobleme.

Image:



```
1
2 FROM node:latest
3 WORKDIR /app
4 COPY . .
5 RUN npm install
6 CMD ["node", "server.js"]
7
```

Answer/Solution:

Verwendung von latest → unsicher / nicht reproduzierbar
Container läuft als root → Sicherheitsrisiko
Keine Kontrolle über Abhängigkeiten (npm install ohne Lockfile)